Developing a Software for Crew Scheduling

Rupesh Anil Sonawane
Roll No.: 23M1533

Department of Industrial Engineering and Operations Research
Indian Institute of Technology Bombay

Supervisor: Prof. Ashutosh Mahajan

Problem Overview

Crew Scheduling Problem

Given a set of train services, determine crew duties such that:

- Every service is covered exactly once.
- All operational constraints are satisfied.
- The total number of duties is minimized.

Application

- Crew scheduling for the Delhi Metro Pink Line Main Loop.
- Generate feasible crew duties from train services.
- Select an optimal set of duties covering all services.

Serv.	Rake	Start Stn	Start	End Stn	End	Dir.	Time	Juris.
577	717	MUPR DN	05:30	MUPR	06:20	UP	50	Yes
11	712	SAKP	05:45	KKDA DN	06:59	DN	74	No

Motivation and Challenges

Challenges in Metro Crew Scheduling

- Large number of train services
- Complex operational regulations
- Multiple feasibility constraints
- Huge combinatorial search space
- Computationally intensive duty generation

Need

- Fast generation of feasible duties
- Efficient optimization framework
- Scalable solution for real-world metro operations

Existing Approach and Limitations

Existing Method

- Recursive Backtracking
- Constraint checking during recursive expansion
- Exploration of a large search space

Limitations

- Repeated feasibility evaluations
- High computational cost
- Poor scalability for large datasets
- Long duty generation time
- Long optimization time

Motivation: Develop a faster and more scalable framework.

Literature Review and Research Gap

- Huisman (2005) demonstrated the use of service connectivity networks for generating feasible crew duties in railway scheduling.
- Kwan (2004) and Caprara et al. (1997) modeled crew scheduling as a Set Partitioning Problem and solved it using Column Generation techniques.
- Existing studies primarily focus on optimization formulations and column generation frameworks for duty selection.
- Limited attention has been given to efficient large-scale feasible duty generation using modern graph traversal and parallel processing techniques.

Research Gap

This work adopts the service-network concept from the literature and develops a graph-based DFS framework with parallel processing to efficiently generate feasible duties before solving the Set Partitioning Problem.

Existing Recursive Backtracking Approach

- Feasible duties were generated by recursively extending one service at a time.
- For every service, all possible successor services were explored using backtracking.
- Operational constraints were repeatedly evaluated during recursive exploration.
- The search process traversed a large number of service combinations sequentially.
- As the number of services increased, the computational effort grew significantly.

Limitation

For the Delhi Metro dataset containing 944 services, feasible duty generation required approximately **5 hours**, making the approach difficult to scale for large instances.

Crew Allocation Rules: Service Connectivity

- Station continuity: End station = Next service start station
- Time feasibility: Next service must start after previous service ends
- Jurisdiction consistency: Duty must start and end in the same jurisdiction
- Rake change allowed only at: KKDA and PVGW
- Same-rake connection: Maximum halt = 15 min (without break)

Crew Allocation Rules: Duty Constraints

- Maximum duty duration: 445 min
- Morning/Evening duty limit: 405 min
- Maximum driving time: 360 min
- Continuous driving limit: 180 min
- Driving time includes: Service time + short gaps ($<$ 30 min)

Crew Allocation Rules: Break Requirements

- Break stations: KKDA or PVGW
- At least one valid break combination required
- Total break time $\leq$ 120 min
- If cont. driving time $>$ 120 min, next break $\geq$ 50 min
- Single break duty: break $\geq$ 50 min
- Multiple breaks: at least one break $\geq$ 50 min
- Non-crew-control start: first break after 90 min

Proposed Framework

Timetable Data $\rightarrow$ Graph Construction $\rightarrow$ Duty Generation $\rightarrow$

Parallel Processing $\rightarrow$ Duty Filtering $\rightarrow$ Set Partitioning $\rightarrow$

Optimal Crew Schedule

Input Service Data

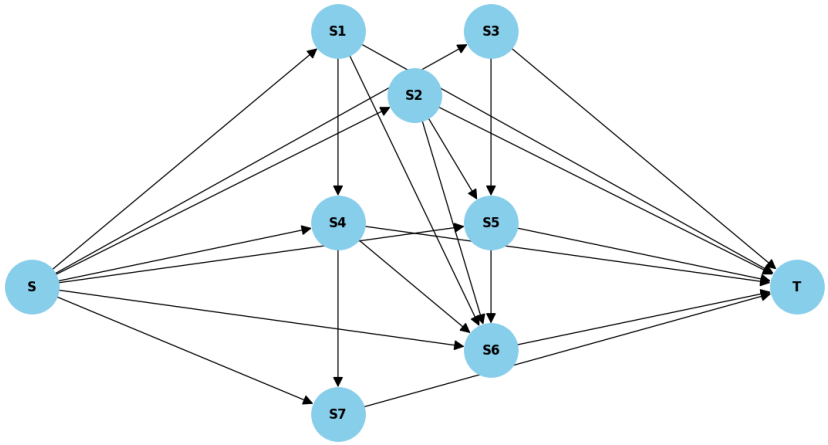
Each train service in the timetable is represented using the following attributes:

Serv.	Rake	Start Stn	Start	End Stn	End	Dir.	Time	Juris.
577	717	MUPR DN	05:30	MUPR	06:20	UP	50	Yes
11	712	SAKP	05:45	KKDA DN	06:59	DN	74	No
570	709	IPE	05:45	PVGW UP	06:57	UP	72	No

- Each row corresponds to a train service.
- Service attributes include timing, stations, rake information, direction, and jurisdiction details.
- In the proposed network model, each service is represented as a node in the graph.

Service Network Representation

- Each node represents a train service.
- Directed edges represent feasible service connections.
- Constraints embedded during graph construction:
  - Station continuity
  - Time feasibility
  - Rake feasibility
  - Rake-change rules
- Source and sink nodes are added for path generation.



Service Network

Nodes	944
Edges	23,234
Rakes	51
Stations	11
Build Time	4 sec

Graph Construction

Node Representation

Each train service is represented as a node.

Edge Creation

A directed edge (S_i, S_j) is added if:

- Station continuity is satisfied
- Time feasibility is satisfied
- Same-rake or valid rake-change condition holds

Benefit

Infeasible service connections are eliminated before path generation, reducing the search space.

Feasible Service Network Construction

Algorithm 1 Construction of Feasible Service Network

```
for all service pairs (s_i, s_j) do
    if station continuity holds then
        if time feasibility holds then
            if rake feasibility holds then
                Add edge (s_i, s_j)
            end if
        end if
    end if
end for
Connect Source and Sink nodes
Return graph G
```

Feasible Duty Generation

Path Enumeration Strategy

- Stack-based Depth-First Search (DFS)
- Starts from the Source node
- Explores all feasible service duties
- Terminates at the Sink node

Advantages

- Eliminates recursive function overhead
- Memory efficient
- Suitable for large service networks
- Enables early pruning of infeasible duties

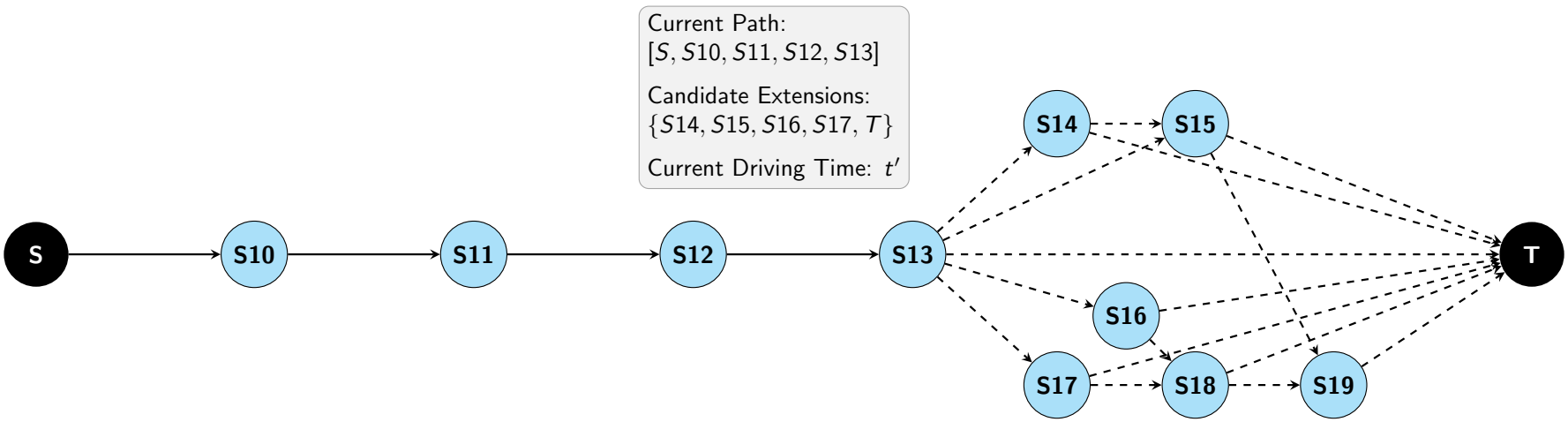
Partial Path Validation algorithm

Algorithm 2 Partial Path Validation

```
1: if driving time exceeds limit then
2:     return FALSE
3: end if
4: if duty duration exceeds limit then
5:     return FALSE
6: end if
7: if continuous driving or morning first break rule is violated then
8:     return FALSE
9: end if
10: return TRUE
```

Path Tracing and Early Pruning

- Feasible duties are generated using stack-based Depth-First Search (DFS).
- Partial paths are validated using driving time, duty duration, and continuous driving constraints.
- Infeasible paths are discarded immediately through early pruning.



Path Tracing in the Service Network

Partial Path Validation

During DFS traversal, each partial duty is validated before further expansion.

Constraint	**Purpose**
Driving Time Limit	Total driving time ≤ 360 min
Duty Duration Limit	Duty duration within allowed limit
Continuous Driving Limit	Continuous driving ≤ 180 min
Morning First-Break Rule	First break after 90 min if required
Long Break Requirement	Break ≥ 50 min after driving > 120 min

Early Pruning

If any constraint is violated, the partial path is discarded immediately and no further expansion is performed.

Final Duty Validation

A duty reaching the Sink node undergoes final feasibility checks.

Constraint	**Requirement**
Jurisdiction Rule	Start and end in same jurisdiction
Valid Break Station	Break at KKDA or PVGW
Jurisdiction Break Rule	At least one break in start jurisdiction
Total Break Duration	≤ 120 min
Single Break Duty	Break ≥ 50 min
Multiple Break Duty	At least one break ≥ 50 min

Final Validation

Only duties satisfying all break and jurisdiction requirements are stored as feasible duties.

Final Duty Validation Algorithm

Algorithm 3 Final Path Validation

```
1:  if jurisdiction constraint violated then
2:      return FALSE
3:  end if
4:  if required break conditions not satisfied then
5:      return FALSE
6:  end if
7:  if cumulative break limit exceeded then
8:      return FALSE
9:  end if
10: if break structure invalid then
11:     return FALSE
12: end if
13: return TRUE
```

Stack-Based DFS Path Enumeration

Algorithm 4 Crew Path Enumeration

```
1: Initialize stack with Source node
2: while stack not empty do
3:     Expand current path
4:     if partial path violates constraints then
5:         Discard path
6:     else if Sink node reached then
7:         Do final validation and save duty
8:     else
9:         Push feasible extensions onto stack
10:    end if
11: end while
```

Constraint Validation and Early Pruning

Graph Construction

- Station continuity
- Time feasibility
- Rake feasibility
- Rake-change rules

Final Duty Validation

- Break requirements
- Jurisdiction constraint

Path Validation

- Duty duration limit
- Driving time limit
- Continuous driving limit
- Morning first-break rule

Key Idea

Infeasible partial duties are discarded immediately, reducing the search space significantly.

Example of a Valid Duty

Table: Duty Timeline

Service	Start	End	From	To	Time	Gap	Break	Rake Change
18	06:15	06:21	VND	KKDA	6	0	–	–
583	06:21	06:36	KKDA	MUPR	15	4	–	–
586	06:40	06:55	MUPR	KKDA	15	0	–	–
28	06:55	08:01	KKDA	PVGW	66	45	45 min	Yes
70	08:46	09:52	PVGW	KKDA	66	51	51 min	Yes
119	10:43	11:50	KKDA	PVGW	67	0	–	–
442	11:50	12:25	PVGW	PVGW	35	0	–	–
159	12:25	13:31	PVGW	KKDA	66	–	–	–

Table: Constraint Validation

Constraint	Condition	Status
Driving Time	$340 \leq 360$	OK
Break Duration	$96 \leq 120$	OK
Duty Duration	$436 \leq 445$	OK
Continuous Drive	$168 \leq 180$	OK
Jurisdiction Match	Valid	OK
Rake Changes	2	Valid

Parallel Processing Framework

- All services are placed into a shared task queue.
- Each worker process retrieves one service from the queue.
- The selected service becomes the first service after the Source node.
- Every worker has access to the complete service network.
- Workers independently perform DFS-based path generation.
- Partial and final path validation are executed locally.

Task Queue $\rightarrow$ Workers $\rightarrow$ Independent DFS $\rightarrow$ Worker CSVs $\rightarrow$ Merged Duty Pool

Parallel Processing Algorithm

Algorithm 5 Parallel Duty Path Enumeration

```
1: Add all first-level services to task queue
2: Create N worker processes
3: for all workers in parallel do
4:     while task queue is not empty do
5:         Retrieve a service from queue
6:         Run DFS starting from that service
7:         Store valid duties in local file
8:     end while
9: end for
10: Merge all worker files
11: Return complete duty dataset
```

Computational Improvement

Configuration	Duty Generation Time
Single Core	27.00 min
10 Cores	2.50 min

$$\text{Speedup} = \frac{27}{2.5} = 10.8 \text{ times}$$

$$\text{Time Reduction} = \left(1 - \frac{2.5}{27}\right) \times 100 = 90.74\%$$

Observation

The parallel implementation achieved an approximately 10.8× speedup and reduced feasible duty generation time by 90.74% while preserving completeness of path enumeration.

Feasible Duty Pool Reduction Heuristic

- A total of 672,036 feasible duties were generated during path enumeration.
- Among these, 24,725 duties contained fewer than four services and were identified as short duties.
- These duties were excluded before solving the set partitioning model.
- Similar short duties were not selected in the optimal solution obtained using the previous methodology.

Total Feasible Duties	672,036
Short Duties Removed	24,725
Remaining Duties	647,311
Optimal Duties Selected	114

Observation

After removing 24,725 short duties, the optimal solution still required only **114 duties**, indicating that the excluded duties were not necessary for achieving optimality.

Set Partitioning Optimization Model

Objective

Select the minimum number of duties such that every service is covered exactly once.

Decision Variable

$$x_j = \begin{cases} 1, & \text{if duty } j \text{ is selected} \\ 0, & \text{otherwise} \end{cases}$$

Objective Function

$$\min \sum_{j \in D} x_j$$

Coverage Constraint

$$\sum_{j \in D} a_{ij} x_j = 1, \qquad \forall i \in S$$

$a_{ij} = 1$ if service i is covered by duty j, otherwise 0.

Optimization Framework

Feasible Duties

(647,311 Duties)

↓

Set Partitioning Model

↓

Minotaur Solver

↓

Optimal Crew Schedule

(114 Duties)

Optimization Results

Total Services	944
Feasible Duties	647,311
Mathematical Solver	Minotaur
Optimal Duties	114
Optimization Time	7.78 min
Service Coverage	100%

Result

The optimization model successfully generated a crew schedule consisting of 114 duties covering all 944 services.

Comparison with Existing Approach

Metric	**Existing Approach**	**Proposed Approach**
Path Generation Method	Recursive Backtracking	Graph-Based DFS
Parallel Processing	No	Yes (10 Cores)
Feasible Duties Generated	672,036	672,036
Duty Generation Time	$\sim$4.5 Hours	2.5 Minutes
Optimization Time	$\sim$2 Hours	7.78 Minutes
Optimal Duties	114	114

Key Observation

The proposed framework achieves the same optimal solution while substantially reducing computational time.

Overall Computational Improvement

Component	Existing	Proposed	Reduction
Graph Creation	–	4 Sec	–
Duty Generation	4.5 Hours	2.5 Min	90.74%
Optimization	2 Hours	7.78 Min	93.5%

Final Outcome

Same optimal solution (**114 duties**) achieved with substantially lower computational time.

Computing Resources

Server	Passpoli Server
Processor	AMD Ryzen Threadripper PRO 5975WX
Cores	32
RAM	128 GB

Note

All duty generation and optimization experiments were performed on the same computing environment to ensure consistent performance evaluation.

Conclusions

- Developed a graph-based framework for metro crew scheduling.
- Generated feasible duties using stack-based DFS with early pruning.
- Reduced duty generation time through parallel processing.
- Applied duty-pool reduction to improve optimization efficiency.
- Solved the resulting Set Partitioning Problem using Minotaur.
- Obtained an optimal crew schedule consisting of 114 duties covering all 944 services.

Summary

The proposed framework achieves the same solution quality as the existing approach while significantly reducing computational effort.

Key References

- **Heil et al. (2020)**
  Comprehensive review of railway crew scheduling models and solution approaches.
- **Desrochers & Soumis (1989)**
  Set Partitioning and Column Generation for urban transit crew scheduling.
- **Caprara et al. (1997)**
  Optimization models and algorithms for railway crew management.
- **Huisman et al. (2005)**
  Service-network representation in railway operations planning.
- **Jin et al. (2022)**
  Network-based crew scheduling for urban rail transit systems.
- **Mahajan et al. (2021)**
  Minotaur optimization toolkit used for solving the set partitioning model.

Thank You

Questions?